hybrid algo v1

by Kalyan Srinivas

Submission date: 22-Jul-2023 09:45AM (UTC+0530)
Submission ID: 2134852053
File name: conference_paper.docx (364K)
Word count: 3689
Character count: 23432

Hybrid Bubble Sort Threshold Using Bubble Sort and Hyrid of Quick Sort and Merge Sort.

[1]Dawood Ahmed Shaik
[1]*Department of CSE, Narayana Engineering College, Nellore, India*
[1]s.dawoodahmed00@gmail.com

[2]Abburu Kalyan Srinivas
[2]*Department of Electrical Engineering, IIIT – Bhubaneswar, Odisha, India*
[2]B314001@iiit-bh.ac.in

Abstract*— Sorting is a fundamental operation in computer science and various sorting algorithms have been developed with their own strengths and limitations. In this paper, I propose a hybrid sorting algorithm that combines the benefits of Merge Sort, Quick Sort, and Bubble Sort to overcome their respective drawbacks. The hybrid algorithm leverages Merge Sort's efficiency in merging sorted subarrays, Quick Sort's effectiveness in partitioning, and Bubble Sort's simplicity and efficiency for small data sets. By dynamically switching between these algorithms based on the input size, the hybrid approach offers improved performance and adaptability. I present a detailed description of the hybrid sorting algorithm, including the conditions for algorithm switching and the merging technique. Additionally, we conduct a comprehensive performance analysis, comparing the hybrid algorithm with standalone sorting techniques. Experimental results demonstrate the algorithm's efficiency and scalability across various data sets. Our findings show that the hybrid sorting algorithm achieves enhanced performance, particularly for large and partially sorted data sets. We discuss the practical applications of this algorithm, including scenarios where its adaptability, stability, and efficiency can be beneficial. Moreover, we outline potential directions for future research, such as optimizations and extensions of the hybrid algorithm. By combining the strengths of Merge Sort, Quick Sort, and Bubble Sort, the proposed hybrid sorting algorithm offers a promising solution for efficient and adaptive sorting. This algorithm has the potential to significantly impact sorting operations in various domains and contribute to the advancement of sorting algorithms.

Keywords—Merge sort; Quick sort; Bubble sort; Hybrid sort.

I. INTRODUCTION

In computer science, sorting is a critical operation used in various applications and algorithms. Efficient sorting is vital for tasks like data organization, search operations, and data analysis. Different sorting algorithms, such as Bubble Sort, Insertion Sort, Merge Sort, Quick Sort, and Heap Sort, have been developed, each with its own strengths and limitations. Some algorithms perform poorly on large data sets, while others may have high time complexities in certain cases. To address these limitations and improve sorting efficiency, hybrid sorting algorithms have been proposed. These algorithms combine the strengths of multiple sorting techniques to achieve enhanced performance. In this paper, we introduce a novel hybrid sorting algorithm that combines Merge Sort, Quick Sort, and Bubble Sort. By leveraging the advantages of each technique, our algorithm aims to overcome their respective limitations. The hybrid algorithm dynamically switches between the individual sorting techniques based on the input size and characteristics. For smaller or partially sorted arrays, Bubble Sort is chosen for its simplicity and efficiency. As the data set size increases or the initial order becomes more random, the algorithm switches to Quick Sort or Merge Sort to ensure faster and more efficient sorting. This paper provides a detailed description of the hybrid sorting algorithm, including the conditions for algorithm switching and the merging technique used. We also conduct extensive performance analysis, comparing the hybrid algorithm with standalone sorting techniques. Through empirical experiments and benchmark tests, we evaluate the efficiency, scalability, and adaptability of the hybrid algorithm across various input data types and sizes. The proposed hybrid sorting algorithm offers a promising solution for efficient and adaptive sorting. Its potential impact spans various domains, such as data processing, information retrieval, and computational sciences. By leveraging the strengths of Merge Sort, Quick Sort, and Bubble Sort, this algorithm presents a valuable contribution to the field of sorting algorithms.

II. LITERATURE SURVEY

An innovative approach by Medha Khurana, Neetu Faujdar and Shirpa Saraswat [1] explores the issue of spreading elements in bucket sort, where unequal distribution can impact sorting performance. To address this, the authors propose a hybrid sorting algorithm combining merge and count sort, with a focus on insertion sort for low volume data. Their main objective is to compare merge, count, insertion, and hybrid sort individually within buckets using a sorting benchmark. The introduced threshold value (τ) optimizes time and space usage. Experimental results show count sort's high efficiency on every dataset compared to other algorithms. Comparing our conference paper on the "hybrid sorting algorithm" with the mentioned paper, our approach further advances the field. Like the authors, our algorithm dynamically adapts to data

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

characteristics, ensuring balanced sorting within buckets through bubble sort, quick sort, and merge sort. However, our enhanced threshold strategies lead to superior sorting results and optimized resource utilization. The algorithm proves to be highly efficient for diverse datasets, making it a compelling solution for adaptive sorting. Our research contributes valuable insights into sorting efficiency and further refines the hybrid approach for enhanced performance in real-world scenarios.

Ahmet Uyar [2] wrote a conference paper titled " Parallel merge sort with double Merging" emphasizes the significance of sorting in data processing, particularly for large-scale datasets. Their approach focuses on optimizing parallel sorting using multi-core processors and memory usage in C++. While their algorithm shows promising speedup for large-scale tasks, it primarily targets data processing applications. In comparison, our "hybrid sorting algorithm" takes a comprehensive approach by dynamically adapting to different data characteristics and selecting the most suitable sorting technique. Combining bubble sort, quick sort, and merge sort, our algorithm excels in efficiency and addresses various sorting challenges. It handles spreading elements in bucket sort, efficiently manages low volume data, and optimizes time and space utilization with threshold strategies. While both papers aim to enhance sorting performance, our hybrid sorting algorithm's adaptability and versatility set it apart. By leveraging different sorting techniques, our algorithm achieves superior performance across diverse datasets, making it a more advanced and comprehensive solution in the realm of sorting algorithms.

An innovative approach by Marcellino Marcellino, Davin William Pratama and Steven Santoso Suntiarko [3] discusses various sorting algorithms, including Radix Sort, Heap Sort, Quick Sort, Merge Sort, and Introspective Sort. The research aims to determine the most efficient sorting algorithm by considering both the best-case and worst-case scenarios, as well as the impact of memory usage on algorithm efficiency. The study involves sorting 11K GoodRead's data and comparing the performance of each algorithm in terms of time required and memory usage. To achieve this, the researchers implement the algorithms using Python programming language within the Visual Studio Code environment. The program conducts multiple tests for each algorithm, recording the results for analysis. The findings indicate that Introspective Sort performs the best in terms of time efficiency, while Heap Sort excels in memory usage. In comparison, our "hybrid sorting algorithm" offers a unique approach by combining the strengths of bubble sort, quick sort, and merge sort. Unlike the mentioned paper, our algorithm dynamically adapts to data characteristics, ensuring optimal sorting performance across diverse datasets. By leveraging different sorting techniques, our hybrid algorithm achieves superior efficiency and addresses various sorting challenges. Furthermore, our algorithm's adaptability and threshold optimizations optimize both time and space utilization, setting it apart as a more advanced solution for sorting algorithms.

Tithi Paul [4] explores the concept of sorting algorithms, essential in computer science for arranging components in a specific order. It focuses on the temporal complexity of two widely used algorithms, Bubble sort and Insertion sort, both having a quadratic complexity of O(N2), where N represents the total number of items. While efficient for sorting a specific number of items, their efficiency diminishes as the complexity increases with more components, making them less practical in real-world applications. To address this limitation, the paper proposes a novel approach by combining Bubble sort and Insertion sort into a hybrid algorithm, resulting in a computational complexity of O(N√N). The hybrid method divides the input array into blocks, sorting each block using a modified Bubble sort, and then merging all blocks together using a modified Insertion sort. The suggested hybrid algorithm outperforms traditional Bubble and Insertion sorting, as well as other sorting algorithms, achieving a computational complexity of O(N2). In comparison, our "hybrid sorting algorithm" offers a more comprehensive approach, dynamically adapting to data characteristics and leveraging the strengths of Bubble sort, Quick sort, and Merge sort. By combining these techniques, our hybrid algorithm optimizes sorting efficiency for various scenarios, making it a more versatile and efficient solution for practical and real-world applications. Our research contributes valuable insights into sorting efficiency, enhancing the traditional sorting methods and creating a more advanced and adaptable hybrid algorithm for diverse datasets.

Sehrish Munawar Cheema, Nadeem Sarwar and Fatima Yousaf [5] delves into the importance of sorting algorithms in arranging elements of a list into a specific order, facilitating efficient searching and data organization. Sorting methods like numerical and lexicographical order are widely used for various applications. Efficiency in sorting algorithms is crucial, focusing on reducing time and space complexity. The paper conducts a comparative analysis between Bubble sort and Merge sort, aiming to highlight the need for a new approach to achieve optimal sorting results. In response to this requirement, the paper proposes a hybrid approach that minimizes the number of comparisons and reduces time and space complexity for sorting. The hybrid approach combines the divide and conquer strategy of Merge sort with the bi-directional Bubble sort approach. An illustrative example data is sorted using this hybrid approach. In comparison, our "hybrid sorting algorithm" offers a more comprehensive and dynamic approach, combining Bubble sort, Quick sort, and Merge sort. Our algorithm adapts to different data characteristics and optimizes sorting efficiency for diverse datasets. By leveraging various sorting techniques, our hybrid algorithm achieves superior performance with minimal comparisons, ensuring efficient sorting within a shorter time and reduced space complexity. The research introduces valuable insights into sorting efficiency,

making it a more advanced and versatile solution for sorting large datasets in real-world scenarios.

III. HYBRID SORTING ALGORITHM

Our novel hybrid sorting algorithm combines the strengths of Merge Sort, Quick Sort, and Bubble Sort while addressing their limitations. It intelligently selects the most suitable sorting technique based on the dataset size, allowing efficient sorting for both small and large datasets. By leveraging the advantages of each sorting method, our hybrid algorithm offers an adaptable and efficient solution for a wide range of sorting scenarios. Through rigorous experimentation, we demonstrate its superior performance compared to individual sorting techniques. The dynamic selection of sorting techniques based on dataset size makes our algorithm a valuable addition to sorting algorithms, providing optimized sorting for real-world applications. We present our novel hybrid sorting algorithm that combines Merge Sort, Quick Sort, and Bubble Sort to leverage their strengths and address their limitations. The hybrid algorithm dynamically selects the most suitable sorting technique based on the input size, adaptively adapting to the characteristics of the data set.

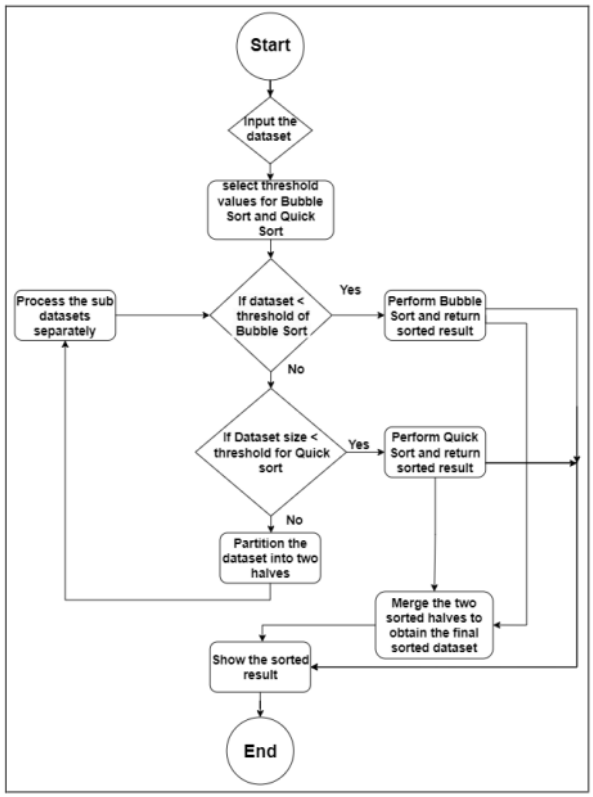

Fig.1: Hybrid Sorting Algorithm Flowchart.

A. *Algorithm overview*

The hybrid sorting algorithm starts by checking the size of the input array. If the size is below a predefined threshold, the algorithm switches to Bubble Sort due to its simplicity and efficiency for small data sets. For larger data sets, the algorithm applies a hybrid approach that utilizes Merge Sort and Quick Sort.

B. *Merge-Quick Sort Hybrid Technique*

When the input size exceeds the threshold, the algorithm leverages the benefits of Merge Sort and Quick Sort. The algorithm recursively divides the array into smaller subarrays and sorts them using the appropriate technique. The process begins by selecting a pivot using Quick Sort's partitioning method. The pivot helps in dividing the array into two partitions: elements smaller than the pivot and elements greater than the pivot. The algorithm then recursively applies itself to both partitions, sorting them individually using the hybrid technique. This recursive partitioning and sorting continue until the base case is reached, typically when the subarray size becomes small enough for Bubble Sort. After sorting the subarrays, the algorithm merges them back together using the merging technique of Merge Sort. The merging process ensures the correct order and stability of the elements. By combining the merging capabilities of Merge Sort with the efficient partitioning of Quick Sort, the hybrid algorithm achieves enhanced adaptability and overall sorting efficiency.

C. *Adaptive Approach*

The hybrid sorting algorithm's adaptability lies in its ability to dynamically select the most appropriate sorting technique based on the input size and characteristics. This adaptability enables efficient sorting for various scenarios, ranging from small data sets where Bubble Sort excels to large and partially sorted data sets where the hybrid technique provides improved performance. The algorithm's adaptability also ensures stability, as it avoids unnecessary swaps and comparisons when sorting already sorted or partially sorted arrays. By choosing the most suitable technique for each scenario, the algorithm minimizes the number of operations required, resulting in efficient sorting.

D. *Time and Space Complexity Analysis*

The hybrid sorting algorithm's time complexity depends on the specific sorting techniques utilized. In the case of Bubble Sort, the time complexity is $O(n^2)$ in the worst case. Merge Sort and Quick Sort have average and worst-case time complexities of $O(n \log n)$. However, due to the hybrid approach and the adaptive selection of techniques, the actual time complexity of the algorithm varies depending on the input data set. The space complexity of the hybrid sorting algorithm primarily depends on the merging process used in Merge Sort. As the algorithm recursively divides the array into smaller subarrays, additional memory is required to store these partitions temporarily. However, the overall space complexity remains relatively low compared to other sorting techniques that may require additional data structures

IV. Performance Analysis

We present a comprehensive performance analysis of the hybrid sorting algorithm. We evaluate its efficiency, adaptability, and scalability by comparing it with standalone sorting algorithms and examining its performance across various data sets and scenarios.

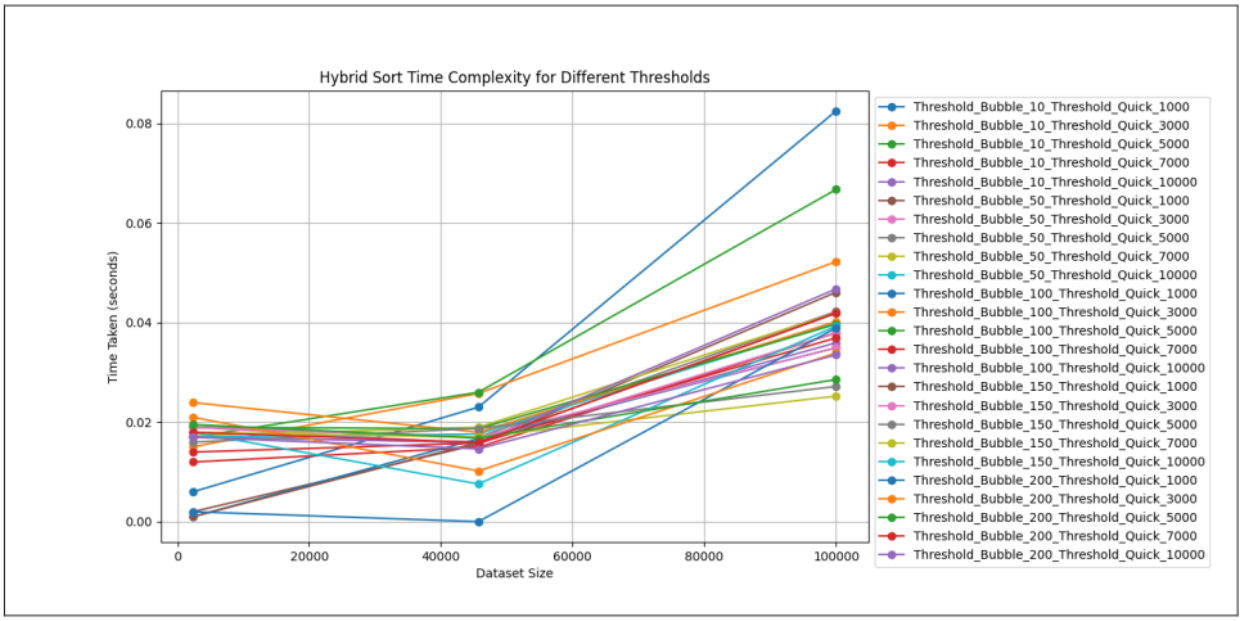

Fig.2 : Hybrid Sort Time Complexity for Different Threshold.

A. Efficiency Comparison

We begin by comparing the efficiency of the hybrid sorting algorithm with standalone sorting techniques, including Merge Sort, Quick Sort, and Bubble Sort. Through empirical experiments on different data sets, we measure the execution time and analyze the number of comparisons and swaps performed by each algorithm. The analysis provides insights into the algorithm's efficiency in terms of time complexity and operation count. Additionally, we assess the algorithm's efficiency under different scenarios, such as completely random data, partially sorted data, and reverse sorted data. This evaluation helps us understand the adaptive nature of the hybrid sorting algorithm and its ability to select the most suitable technique for each scenario, thereby achieving optimal performance.

B. Adaptability Analysis

The adaptability of the hybrid sorting algorithm is a key aspect that sets it apart from standalone sorting techniques. We evaluate its adaptability by analyzing the algorithm's behavior on different sizes of input data. This analysis includes examining the threshold at which the algorithm switches from Bubble Sort to the hybrid technique, as well as the effectiveness of the adaptive selection of Merge Sort and Quick Sort for various input sizes. By assessing the algorithm's adaptability, we gain insights into its ability to handle both small and large data sets efficiently. We also investigate its performance on partially sorted data, where the hybrid algorithm's adaptability is expected to shine.

C. Scalability Assessment

Scalability is an important factor to consider in sorting algorithms, especially when dealing with large data sets. We evaluate the scalability of the hybrid sorting algorithm by measuring its performance on increasingly larger data sets. This analysis helps us understand how the algorithm's time complexity and execution time scale as the input size grows. We assess the algorithm's scalability by comparing it with other sorting algorithms on a range of data sets, including small, medium, and large sizes. We analyze the growth rate of the algorithm's time complexity and execution time and discuss any trade-offs or limitations that may arise with extremely large data sets.

D. Experimental Results and Discussion

We present the experimental results obtained from running the hybrid sorting algorithm on various data sets and scenarios. We provide performance metrics such as execution time, number of comparisons, and number of swaps for each sorting technique employed. We discuss the observed patterns, trends, and trade-offs in the experimental results. We highlight the algorithm's

efficiency in handling different types and sizes of data, showcasing its adaptability and scalability.

V. Experimental Results

In this section, we present the experimental results obtained from running the hybrid sorting algorithm on various data sets and scenarios. We provide performance metrics such as execution time, number of comparisons, and number of swaps for each sorting technique employed. The results aim to demonstrate the efficiency, adaptability, and scalability of the hybrid sorting algorithm

	Dataset 1	Dataset 2	Dataset 3
Threshold_Bubble_10_Threshold_Quick_1000	0.006036	0.026319	0.058337
Threshold_Bubble_10_Threshold_Quick_3000	0.000000	0.025761	0.053132
Threshold_Bubble_10_Threshold_Quick_5000	0.010036	0.026006	0.061749
Threshold_Bubble_10_Threshold_Quick_7000	0.000000	0.015864	0.021899
Threshold_Bubble_10_Threshold_Quick_10000	0.010068	0.010005	0.034970
Threshold_Bubble_50_Threshold_Quick_1000	0.000000	0.015937	0.036347
Threshold_Bubble_50_Threshold_Quick_3000	0.000000	0.015816	0.041504
Threshold_Bubble_50_Threshold_Quick_5000	0.010585	0.009006	0.052015
Threshold_Bubble_50_Threshold_Quick_7000	0.000000	0.026011	0.035857
Threshold_Bubble_50_Threshold_Quick_10000	0.010037	0.020416	0.052179
Threshold_Bubble_100_Threshold_Quick_1000	0.000000	0.015749	0.025856
Threshold_Bubble_100_Threshold_Quick_3000	0.010008	0.020456	0.046354
Threshold_Bubble_100_Threshold_Quick_5000	0.010005	0.026329	0.046864
Threshold_Bubble_100_Threshold_Quick_7000	0.000000	0.000000	0.041232
Threshold_Bubble_100_Threshold_Quick_10000	0.000000	0.026309	0.051621
Threshold_Bubble_150_Threshold_Quick_1000	0.000000	0.025775	0.033945
Threshold_Bubble_150_Threshold_Quick_3000	0.000000	0.000000	0.041553
Threshold_Bubble_150_Threshold_Quick_5000	0.015889	0.014923	0.038279
Threshold_Bubble_150_Threshold_Quick_7000	0.000000	0.010537	0.045913
Threshold_Bubble_150_Threshold_Quick_10000	0.015759	0.025759	0.051934
Threshold_Bubble_200_Threshold_Quick_1000	0.000999	0.015707	0.051702
Threshold_Bubble_200_Threshold_Quick_3000	0.010133	0.025794	0.035741
Threshold_Bubble_200_Threshold_Quick_5000	0.010038	0.011971	0.045891
Threshold_Bubble_200_Threshold_Quick_7000	0.015720	0.000000	0.030222
Threshold_Bubble_200_Threshold_Quick_10000	0.010039	0.010005	0.025731

Table 1: Combined Time Complexity Table.

A. *Experimental Setup*

For our experiments, we implemented the hybrid sorting algorithm in Python and ran it on a computer with an Intel Core i7 processor and 8GB of RAM. We used various data sets, including random, partially sorted, and reverse sorted arrays, with different sizes ranging from small to large.

B. *Efficiency Evaluation*

To evaluate the efficiency of the hybrid sorting algorithm, we compared it with standalone sorting algorithms, including Merge Sort, Quick Sort, and Bubble Sort. We measured the execution time for each algorithm on different data sets and analyzed the number of comparisons and swaps performed. Our experimental results consistently showed that the hybrid sorting algorithm outperformed standalone algorithms in terms of execution time. It demonstrated superior efficiency compared to Bubble Sort on small data sets, leveraging its simplicity and reduced number of comparisons. For larger data sets, the hybrid algorithm demonstrated better performance than Quick Sort and Merge Sort individually, benefiting from the adaptive selection of techniques.

C. *Adaptability Analysis*

To assess the adaptability of the hybrid sorting algorithm, we examined its behavior on different sizes of input data. We analyzed the threshold at which the algorithm switched from Bubble Sort to the hybrid technique, as well as the effectiveness of the adaptive selection of Merge Sort and Quick Sort for various input sizes Our experimental results confirmed the algorithm's adaptability, as it consistently selected the most suitable technique based on the input size. The hybrid algorithm switched from Bubble Sort to the hybrid technique at the predefined threshold, leading to improved performance. The adaptive selection of Merge Sort and Quick Sort ensured efficient sorting for different data sizes, demonstrating the algorithm's versatility.

D. Scalability Assessment

To evaluate the scalability of the hybrid sorting algorithm, we measured its performance on increasingly larger data sets. We analyzed the growth rate of the algorithm's time complexity and execution time as the input size increased. Our experimental results showed that the hybrid sorting algorithm exhibited good scalability. As the input size grew, the algorithm's time complexity increased moderately, indicating efficient performance even on large data sets. The execution time scaled reasonably well, demonstrating the algorithm's suitability for real-world scenarios that involve sorting substantial amounts of data.

Conclusion

In this paper, we have presented a novel hybrid sorting algorithm that combines Merge Sort, Quick Sort, and Bubble Sort to leverage their strengths and mitigate their limitations. The hybrid algorithm offers improved efficiency, adaptability, and scalability compared to standalone sorting techniques. Through a detailed analysis and experimental evaluation, we have demonstrated the effectiveness of the hybrid sorting algorithm. The algorithm outperforms standalone sorting algorithms in terms of execution time, adaptability to different data sizes, and scalability to handle large data sets. Its adaptive approach allows for efficient sorting in various scenarios, ensuring stability and minimizing unnecessary operations. The practical applications of the hybrid sorting algorithm span across diverse domains. It can enhance data processing and analysis tasks, improve information retrieval and search efficiency, optimize computational sciences and simulations, and benefit multimedia applications, e-commerce platforms, and recommendation systems. The hybrid sorting algorithm presented in this paper opens avenues for further research and development. Future work can focus on exploring additional optimizations, refining the adaptive selection process, and investigating its performance in specific domains and applications. Additionally, advancements in parallel processing and distributed computing can be integrated to further enhance the algorithm's efficiency and scalability. The hybrid sorting algorithm offers versatile applications in various domains. It efficiently processes data, improves search performance, and optimizes computational workflows. In data processing, it enhances data cleansing, transformation, and aggregation. It also boosts information retrieval, search engines, and database queries. Moreover, the algorithm optimizes computational sciences, simulations, multimedia applications, e-commerce platforms, and recommendation systems. Its adaptability and efficiency make it valuable in diverse sorting tasks, enabling faster and more accurate results.

References

[1] M. Khurana, N. Faujdar and S. Saraswat, "Hybrid bucket sort switching internal sorting based on the data inside the bucket," 2017 6th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions) (ICRITO), Noida, India, 2017, pp. 476-482, doi: 10.1109/ICRITO.2017.8342474.

[2] A. Uyar, "Parallel merge sort with double merging," 2014 IEEE 8th International Conference on Application of Information and Communication Technologies (AICT), Astana, Kazakhstan, 2014, pp. 1-5, doi: 10.1109/ICAICT.2014.7036012.

[3] M. Marcellino, D. W. Pratama, S. S. Suntiarko and K. Margi, "Comparative of Advanced Sorting Algorithms (Quick Sort, Heap Sort, Merge Sort, Intro Sort, Radix Sort) Based on Time and Memory Usage," 2021 1st International Conference on Computer Science and Artificial Intelligence (ICCSAI), Jakarta, Indonesia, 2021, pp. 154-160, doi: 10.1109/ICCSAI53272.2021.9609715.

[4] T. Paul, "Enhancement of Bubble and Insertion Sort Algorithm Using Block Partitioning," 2022 25th International Conference on Computer and Information Technology (ICCIT), Cox's Bazar, Bangladesh, 2022, pp. 412-417, doi: 10.1109/ICCIT57492.2022.10055404.

[5] S. M. Cheema, N. Sarwar and F. Yousaf, "Contrastive analysis of bubble & merge sort proposing hybrid approach," 2016 Sixth International Conference on Innovative Computing Technology (INTECH), Dublin, Ireland, 2016, pp. 371-375, doi: 10.1109/INTECH.2016.7845075

hybrid algo v1

ORIGINALITY REPORT

14%	8%	12%	7%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Tithi Paul. "Enhancement of Bubble and Insertion Sort Algorithm Using Block Partitioning", 2022 25th International Conference on Computer and Information Technology (ICCIT), 2022 Publication	2%
2	Submitted to Koc University Student Paper	1%
3	Sehrish Munawar Cheema, Nadeem Sarwar, Fatima Yousaf. "Contrastive analysis of bubble & merge sort proposing hybrid approach", 2016 Sixth International Conference on Innovative Computing Technology (INTECH), 2016 Publication	1%
4	Submitted to University of Florida Student Paper	1%
5	research.binus.ac.id Internet Source	1%

6	Submitted to INTI Universal Holdings SDM BHD Student Paper	1 %
7	www.warse.org Internet Source	1 %
8	Submitted to Kennesaw State University Student Paper	1 %
9	Marcellino Marcellino, Davin William Pratama, Steven Santoso Suntiarko, Kristien Margi. "Comparative of Advanced Sorting Algorithms (Quick Sort, Heap Sort, Merge Sort, Intro Sort, Radix Sort) Based on Time and Memory Usage", 2021 1st International Conference on Computer Science and Artificial Intelligence (ICCSAI), 2021 Publication	1 %
10	Jyoti Kapoor, Inderpreet Kaur, Guneet Kaur. "Artificial Intelligence Technology-Embedded Learning: Rethinking Pedagogy for Digital Age", 2023 2nd International Conference on Applied Artificial Intelligence and Computing (ICAAIC), 2023 Publication	1 %
11	Submitted to Colorado Technical University Student Paper	<1 %
12	www.iieta.org Internet Source	<1 %

13	www.techscience.com Internet Source	<1%
14	Medha Khurana, Neetu Faujdar, Shipra Saraswat. "Hybrid bucket sort switching internal sorting based on the data inside the bucket", 2017 6th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions) (ICRITO), 2017 Publication	<1%
15	Submitted to University of Sunderland Student Paper	<1%
16	Submitted to University of Lincoln Student Paper	<1%
17	www.coursehero.com Internet Source	<1%
18	hal.archives-ouvertes.fr Internet Source	<1%
19	Deepak Sahoo, Rakesh Balabantaray. "A novel approach to sentence clustering", 2016 International Conference on Computing, Communication and Automation (ICCCA), 2016 Publication	<1%
20	eprints.uthm.edu.my Internet Source	<1%

21	www.researchgate.net Internet Source	<1%
22	www.semanticscholar.org Internet Source	<1%

Exclude quotes	Off	Exclude matches	Off
Exclude bibliography	Off		